

Persistência

O que vamos aprender?

A **memória** é **volátil**, o que significa que os dados são perdidos ao reiniciar (ou travar) o computador. Dispositivos de disco (por exemplo, HDDs, SSDs) permitem que os programas armazenem seus dados persistentemente. Vamos aprender:

- Como o SO abstrai o acesso (API) aos **drivers** de baixo nível desses discos?
- Como o SO torna as operações de disco eficientes quando vários programas usam o mesmo dispositivo simultaneamente?
- Normalmente, acessamos nossos dados usando uma abstração baseada em **arquivos** (arquivos, pastas, etc.). Como o SO possibilita essa abstração?

Vamos estudar a interface e o design dos sistemas de arquivos para responder a essas perguntas.

Interfaces de Armazenamento

As mais comuns:

- **Block Device**
 - Os dados são gerenciados como um conjunto de **blocos** (abstração mais próxima do disco).
 - Soluções: Linux block device, iSCSI
- **File System (FS)**
 - Os dados são gerenciados como uma **hierarquia** de diretórios e arquivos (a interface em que a maioria dos usuários confia em seus computadores pessoais).
 - Soluções: Ext4, ZFS, NFS

```
POSIIX API (open, read, write, ...)  
Application  
Generic Block Interface (block read/write)  
Specific Block Interface (protocol-specific)  
File System  
RAW  
User  
Kernel  
Generic Block Layer  
Device Driver (ATA, IDE, SCSI)  
Disk device  
A simplified Linux File System stack
```

- **Key-value Store**
 - Os dados são gerenciados como pares **chave-valor** (ou seja, a chave é um identificador único e o valor é o conteúdo).
 - Soluções: LevelDB, RocksDB

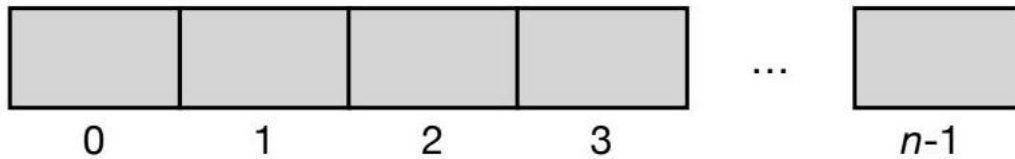
Abstrações do Sistema de Arquivos

O Arquivo

Um arquivo é um array linear de bytes que podem ser lidos e escritos.

- Normalmente, os usuários estão cientes apenas do nome de alto nível legível por humanos de um arquivo (por exemplo, foo.txt)
- Identificado exclusivamente por um nome de baixo nível, o número do **inode** (i-number)
- Os nomes legíveis por humanos são comumente compostos pelo nome do arquivo (por exemplo, foo, bar) e o tipo (por exemplo, .txt, .mp3, .jpg)
 - Geralmente, isso é apenas uma convenção (por exemplo, um arquivo chamado main.c pode não conter código C)
- A maioria dos sistemas operacionais sabe pouco sobre a estrutura do arquivo (por exemplo, se é uma imagem, arquivo de texto, código)

Aqui está uma representação visual de um arquivo como um array de bytes:



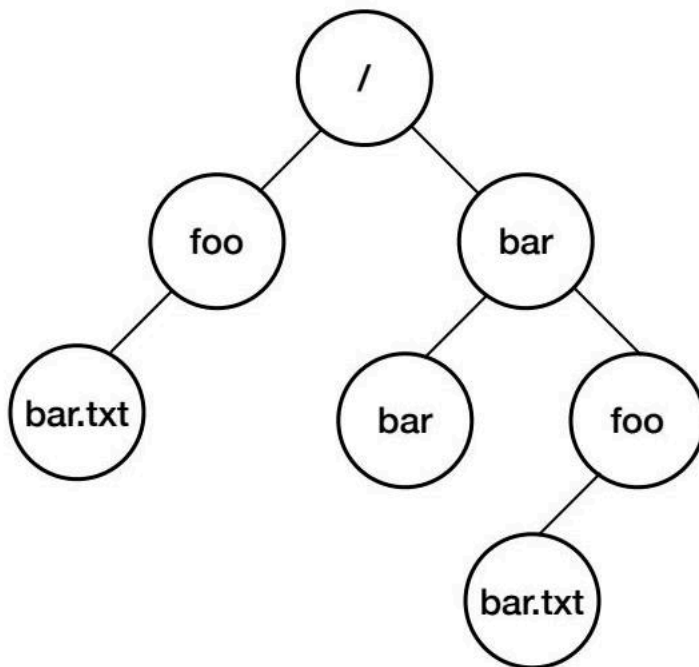
O diagrama mostra um array de elementos numerados de 0 a $n - 1$, representando os bytes do arquivo.

O Diretório

Um diretório é um container de arquivos e outros diretórios, também identificado por um i-number.

- Um diretório contém uma lista de pares, cada um mapeando um nome legível por humanos para seu i-number (ou seja, cada um se referindo a um arquivo ou outro diretório).
- Exemplo: supondo que o arquivo bar.txt tenha i-number 10, o diretório foo onde o arquivo reside tem o par (bar.txt, 10).
- Ao colocar diretórios dentro de diretórios, pode-se construir árvores de diretórios (ou hierarquias de diretórios).
 - A hierarquia começa no diretório raiz (por exemplo, / no UNIX) e usa um separador para nomear subdiretórios subsequentes.
- Um nome de caminho absoluto de arquivo é o caminho completo do diretório raiz até esse arquivo (por exemplo, /bar/foo/bar.txt)
- Arquivos e diretórios podem ter o mesmo nome, desde que estejam em locais diferentes na árvore de diretórios.

A imagem a seguir ilustra uma árvore de diretórios:



Este diagrama ilustra como os diretórios podem ser organizados hierarquicamente, com um diretório raiz que se ramifica em vários subdiretórios e arquivos.

Interface do Sistema de Arquivos

Abrindo e Criando Arquivos

A chamada de sistema `int open(const char *pathname, int oflag [, mode])` permite abrir e criar arquivos:

- **pathname**: o nome do caminho do arquivo legível por humanos. Pode ser um nome de caminho absoluto ou relativo (ao diretório de trabalho atual).
- **oflag**: flags especificando o modo de abertura.
- **mode**: permissões do arquivo. Deve ser usado ao criar um arquivo (por exemplo, 0600 dá permissões para o proprietário do arquivo para lê-lo e escrevê-lo).

Algumas flags interessantes:

- **O_WRONLY, O_RDONLY, O_RDWR**: abre o arquivo para escrita, leitura ou ambos, respectivamente.
- **O_CREAT**: cria o arquivo se ele não existir.
- **O_TRUNC**: trunca o tamanho do arquivo para zero bytes, removendo assim qualquer conteúdo existente.
- **O_APPEND**: anexa o arquivo em cada escrita.

`open` retorna um **file descriptor** (ou -1 em caso de erro).

O File Descriptor (FD)

Um File Descriptor (FD) é um inteiro que pode ser usado para ler e/ou escrever o arquivo.

Pense nisso como um ponteiro para um objeto que outras chamadas de sistema usam para acessar o arquivo. FDs podem representar outros recursos de Entrada/Saída (por exemplo, pipes, outros dispositivos de E/S).

- Cada processo mantém um array de file descriptors (também referido como tabela de file descriptors).
- Quando um processo começa a ser executado, esta tabela de processo já tem FDs padrão inicializados para:
 - O processo para ler a entrada, por exemplo, do teclado (entrada padrão - FD 0).
 - Para escrever a saída na tela (saída padrão - FD 1).
 - Para escrever mensagens de erro (erro padrão - FD 2).

```
FD array of Process A
-----
| 0 | 1 | 2 | ... | Other FDs (non-initialized) | ... |
-----
| stdin | stdout | stderr |
-----
```

Para simplificar, vamos supor que os FDs 0, 1 e 2 apontam para algumas estruturas do SO que realmente contêm as informações para ler do stdin e escrever no stdout e stderr.

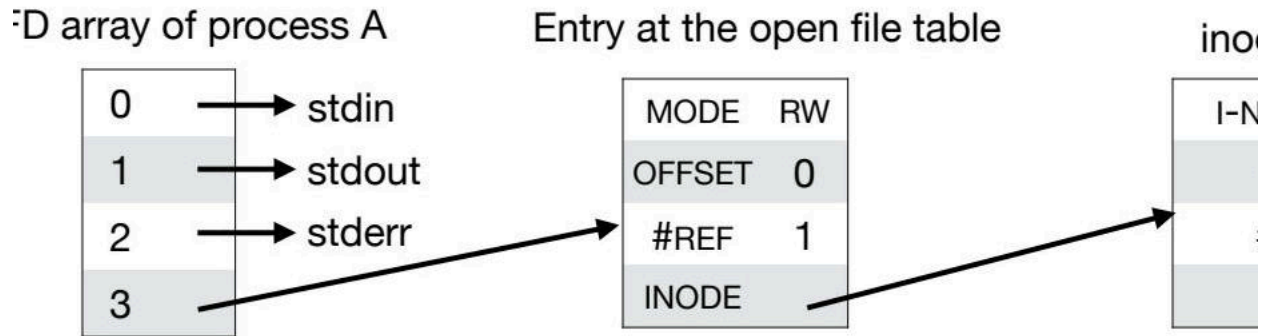
Exemplo: Abrindo um Arquivo

O Processo A executa o código:

```
int fd = open("foo.txt", O_CREAT | O_TRUNC | O_RDWR, 0600);
```

- O file descriptor é inicializado e retornado pela chamada `open()` (ou seja, `fd = 3`).
- Uma entrada na tabela de arquivos abertos em todo o sistema é criada, para a qual o file descriptor 3 aponta.
 - Cada entrada rastreia se o arquivo é legível, gravável ou ambos, o deslocamento do arquivo atual e algumas outras informações.
 - Na maioria dos casos, os processos não compartilham as entradas da tabela de arquivos abertos. Por exemplo, dois processos não relacionados que abrem o arquivo `foo.txt` terão entradas independentes (discutiremos algumas exceções mais adiante).
- A entrada da tabela de arquivos abertos mapeia para uma entrada de inode (identificada por seu i-number).
 - O inode contém, por exemplo, o tamanho do arquivo, juntamente com mais informações que discutiremos mais adiante.

A imagem abaixo resume este processo:



O diagrama mostra como um file descriptor em um processo aponta para uma entrada na tabela de arquivos abertos, que por sua vez aponta para o inode do arquivo.

Escrevendo e Lendo Arquivos

A chamada de sistema `ssize_t write(int fd, const void *buf, size_t nbyte)` permite escrever bytes em um file descriptor:

- **fd**: o file descriptor.
- **buf**: buffer de memória com conteúdo a ser escrito.
- **nbyte**: número de bytes para escrever do buffer.

Retorna o número de bytes escritos ou um erro (-1).

A chamada de sistema `ssize_t read(int fd, void *buf, size_t nbyte)` permite ler bytes de um file descriptor:

- **fd**: o file descriptor.
- **buf**: buffer de memória para onde o conteúdo é lido.
- **nbyte**: número máximo de bytes para ler.

Retorna o número de bytes lidos ou um erro (-1).

Tenha cuidado para não especificar um número de bytes para ler/escrever maior do que a memória alocada para o buffer!

Alternativamente, as gravações e leituras em arquivos podem ser feitas através da chamada de sistema `mmap()`. Ela mapeia o arquivo para a memória para que possa ser manipulado com instruções de memória.

Exemplo: Escrevendo em um Arquivo

O Processo A agora executa uma escrita no arquivo:

```
ssize_t res = write(fd, "abcde", 5);
```

- A chamada `write()` escreve os 5 bytes no arquivo e retorna (ou seja, `res=5`).
- O offset na tabela de arquivos abertos e o tamanho no inode são atualizados.

O diagrama a seguir ilustra o estado do arquivo após a operação de escrita:

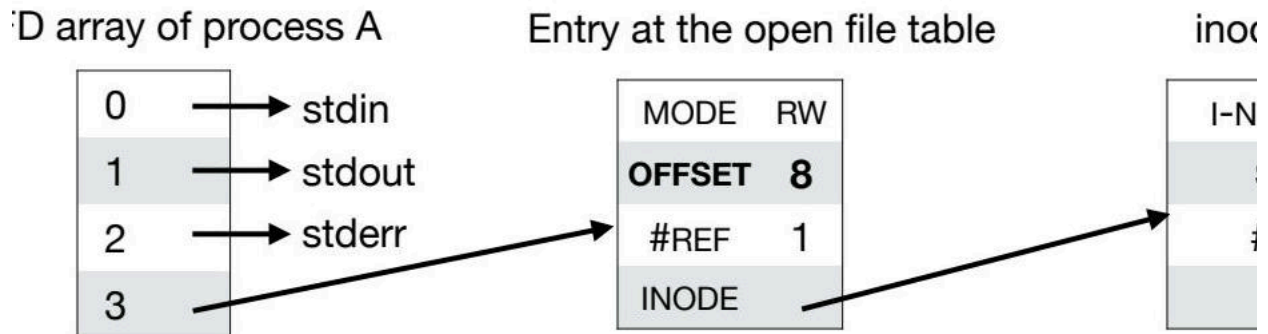


A imagem mostra o conteúdo do arquivo "foo.txt" como "ABCDE" e indica que o offset atual no file descriptor 3 é 5.

Se o Processo A executar outra escrita no arquivo:

```
res = write(fd, "fgh", 3);
```

- A chamada `write()` escreve os 3 bytes no arquivo (começando no offset 5) e retorna (ou seja, `res=3`).
- O offset na tabela de arquivos abertos e o tamanho no inode são atualizados.



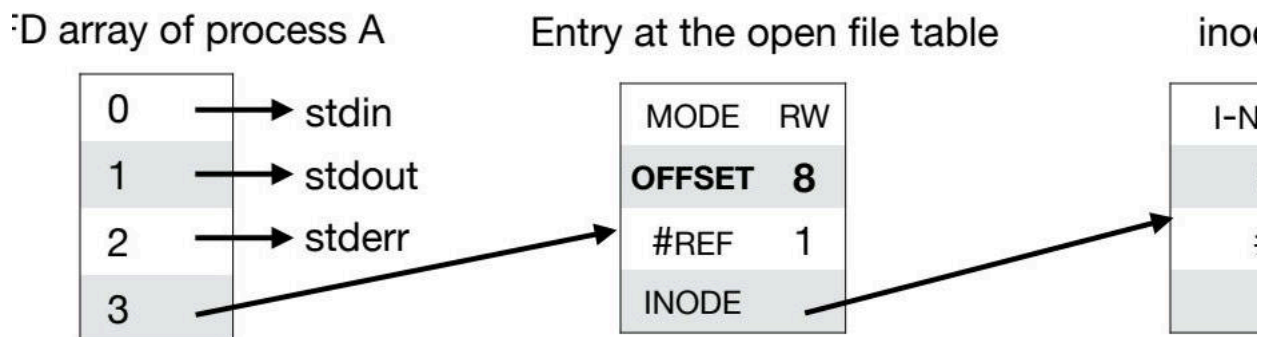
Agora, o conteúdo do arquivo "foo.txt" é "ABCDEFGH", e o offset atual no FD 3 é 8.

Exemplo: Lendo um Arquivo

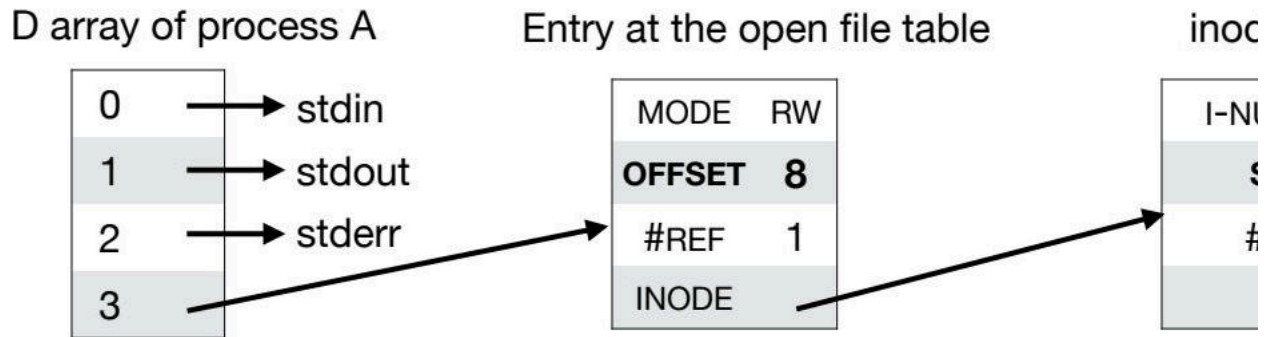
O Processo A agora executa uma leitura do arquivo:

```
res = read(fd, buf, 8);
```

Qual será o valor de retorno da chamada `read`?



Como o offset está posicionado no final do arquivo (offset = 8), a chamada retornará 0, indicando que não há mais nada para ler.



Múltiplos File Descriptors

- Opção 1: Para ler todo o conteúdo do arquivo, pode-se emitir outra chamada `open()`.
 - Por exemplo, executando o código:

```
int fd1 = open("foo.txt", O_RDONLY);
```

```
* A chamada `open()`` criará uma nova entrada na tabela de arquivos abertos que terá seu offset definido como 0.
* Emita a chamada `read()`` para o descritor recém-aberto (ou seja, fd1 = 4).
* Observe que o inode é compartilhado entre as duas entradas da tabela de arquivos.
```

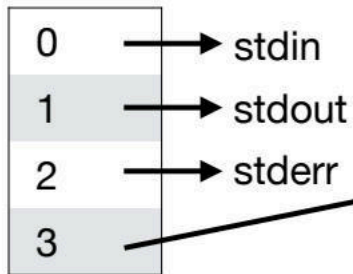
```
Entries at the open file table
-----
| 0 | 1 | 2 | 3 | 4 |
-----
| stdin| stdout| stderr| MODE RW OFFSET 8 #REF 1 INODE | MODE R OFFSET 0 #REF 1 INODE |
-----
inode for file foo.txt
I-NUMBER 100
SIZE 8
#REF 2
```

- Opção 2: Faça um acesso aleatório ao offset 0 do descritor.
 - A chamada de sistema `off_t lseek(int fd, off_t offset, int whence)` permite alterar o offset para um determinado file descriptor.
 - fd**: o file descriptor
 - offset**: o número de bytes para mover para frente ou para trás (pode ser negativo)
 - whence**: o offset base no arquivo ao qual o argumento de offset deve ser adicionado
 - SEEK_SET**: posiciona no início do arquivo e aplica o argumento de offset.
 - SEEK_END**: posiciona no final do arquivo e aplica o argumento de offset.
 - SEEK_CUR**: aplica o argumento de offset na localização atual (offset atual) no arquivo.
 - Retorna o offset resultante no arquivo ou um erro (-1).
 - Observação: `lseek()` não executa uma busca no disco. Ele apenas altera a variável de offset na entrada da tabela de arquivos. Uma busca no disco é executada ao acessar setores em trilhas distintas.

Exemplo: Buscando e Lendo um Arquivo

O processo primeiro executa `lseek(fd, 0, SEEK_SET)`:

FD array of process A



Entry at the open file table

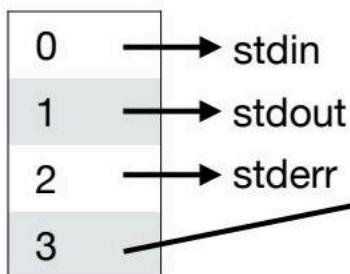
MODE	RW
OFFSET	8
#REF	1
INODE	

ino

I-N

Isso significa que o offset é posicionado no início do arquivo. Agora, pode-se executar a chamada `read()`.

FD array of process A



Entry at the open file table

MODE	RW
OFFSET	0
#REF	1
INODE	

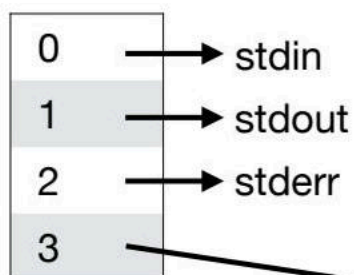
in

I

Compartilhando Entradas da Tabela de Arquivos Abertos com `fork`

- Processos não relacionados que abrem o mesmo arquivo têm suas próprias entradas na tabela de arquivos (o inode é compartilhado).
- Uma exceção: Quando um processo chama `fork()`, o processo filho tem seu próprio array de file descriptors, mas compartilha a entrada da tabela de arquivos abertos com o pai.
 - Cuidado: múltiplos processos relacionados lendo, escrevendo e buscando um arquivo podem atualizar o campo de offset concorrentemente!

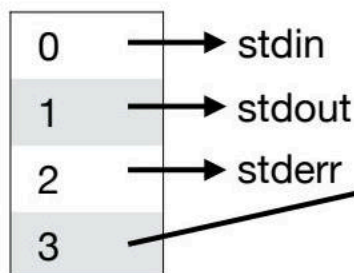
FD array of parent



Entry at the open file table

MODE	RW
OFFSET	0
#REF	2
INODE	

FD array of child

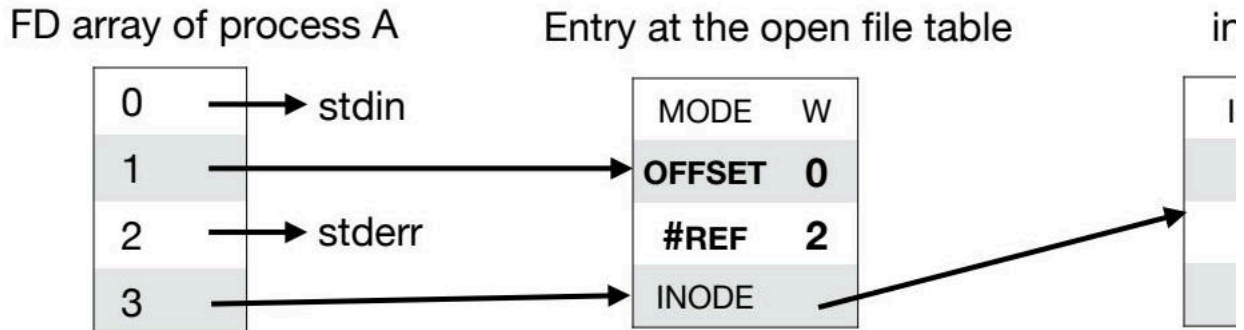


Compartilhando Entradas da Tabela de Arquivos Abertos com `dup`

Outra exceção, onde as entradas da tabela de arquivos são compartilhadas, é ao usar a família de chamadas de sistema `dup()`. Estes duplicam um file descriptor existente, fazendo com que sua cópia aponte para a mesma entrada da tabela de arquivos.

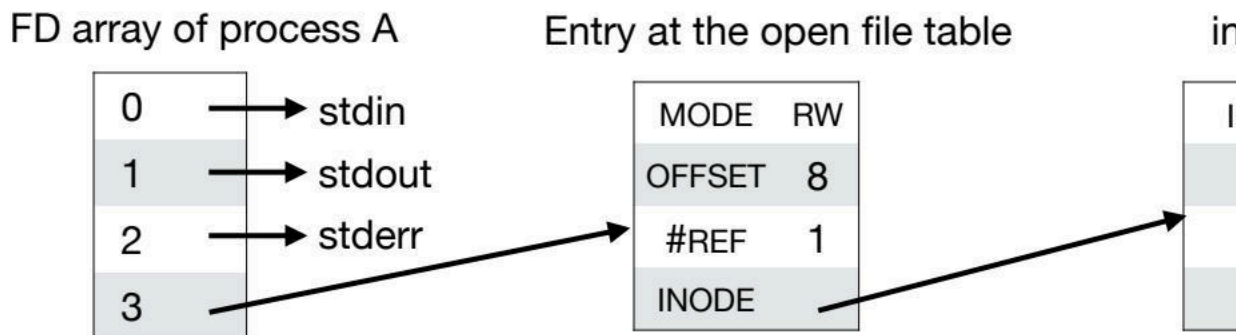
- `dup2(oldfd, newfd)` permite duplicar o file descriptor `oldfd` em `newfd`.

O exemplo abaixo mostra o resultado da execução de `dup2(3,1)`:



Fechando e Removendo Arquivos

- A chamada de sistema `int close(int fd)` "libera" o descritor correspondente (`fd`) no array por processo. Se a entrada na tabela de arquivos não for mais referenciada, ela será excluída.
 - O inode não é excluído!
- A chamada de sistema `int unlink(const char *pathname)` exclui apenas o inode associado ao nome do caminho quando o número de referências a ele chega a 0.
 - Lembre-se de que múltiplos processos não relacionados podem ter o mesmo arquivo aberto.



Descarregando Dados Buffering do Sistema de Arquivos para o Disco

- Quando um programa chama `write()`, ele pede ao sistema de arquivos (FS) para persistir os dados.
- No entanto, o FS armazenará em buffer os dados gravados na memória por algum tempo (por exemplo, 5 segundos, 30 segundos...).
- Alguma ideia do porquê?
- Se a máquina travar nesse meio tempo, os dados em buffer serão perdidos...
- Aplicações como bancos de dados exigem fortes garantias de persistência.
- A chamada `int fsync(int fd)` força o FS a gravar todos os dados sujos de um determinado file descriptor (`fd`) no disco.
- Em alguns casos, também é necessário sincronizar o diretório onde o arquivo reside (ou seja, se o arquivo foi recém-criado, o diretório deve saber que ele existe...) Isso é frequentemente esquecido, levando a bugs...

Renomeando Arquivos

A chamada de sistema `int rename(const char *oldpath, const char *newpath)` renomeia um arquivo (ou seja, altera seu nome legível por humanos).

- É assim que o comando `mv` é implementado.
- Em muitos sistemas de arquivos, essa operação é atômica (ou seja, se o sistema travar durante uma renomeação, o arquivo será nomeado com o nome antigo ou com o novo).
- Muito útil para suportar aplicações que exigem atualizações atômicas para determinados arquivos.
 - Por exemplo, quando os usuários atualizam o texto no meio de um arquivo, alguns editores de texto criam um arquivo temporário para reorganizar o conteúdo. Quando isso é feito, o arquivo temporário é renomeado novamente para o original.

Obtendo Informações do Arquivo

As chamadas de sistema `stat()` e `fstat()` permitem verificar metadados sobre um arquivo, dado seu nome de arquivo ou descritor, respectivamente.

- As informações são fornecidas em uma estrutura `stat`.
- As informações contidas nesta estrutura correspondem às armazenadas no inode do arquivo.

Manipulando Diretórios

- Os diretórios não podem ser gravados. Estes são atualizados indiretamente criando arquivos ou outros diretórios neles.
- `mkdir()` cria um diretório vazio.
- `opendir()` abre o diretório.
- `readdir()` permite ler cada entrada contida dentro do diretório.
- `closedir()` fecha o diretório.
- `rmdir()` exclui o diretório (falha se o diretório não estiver vazio).

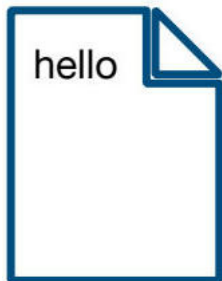
Controle de Acesso

- Arquivos e diretórios são compartilhados entre vários usuários.
- Os bits de permissão (`rw-r--r--`) permitem especificar o que o proprietário do arquivo, alguém em um grupo e outros usuários podem fazer.
 - As permissões podem ser definidas em um formato octal (0644 para o exemplo acima). Explore <https://chmod-calculator.com/> para saber mais.
 - O comando `chmod` pode ser usado para alterar tais permissões.
- Alguns sistemas de arquivos fornecem outros mecanismos de proteção.
 - O AFS, por exemplo, permite definir uma Lista de Controle de Acesso (ACL) por diretório.

Rastreamento de Chamadas de Sistema: A Ferramenta `strace`

`Strace` é um programa para rastrear chamadas de sistema.

- Pode ser usado com qualquer programa, não exigindo seu código-fonte.
- Útil para observar quais serviços do SO o programa está solicitando.



Exemplo:

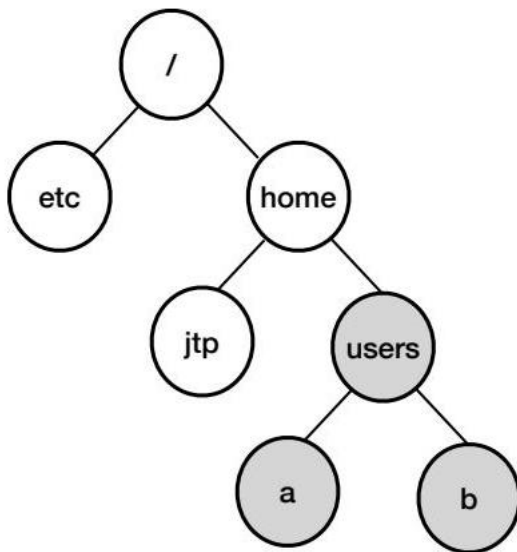
```
prompt> strace cat foo.txt
...
open("foo", O_RDONLY|O_LARGEFILE) = 3
read(3, "hello\n", 4096) = 6
write(1, "hello\n", 6) = 6
...
read(3, "", 4096) = 0
close(3) = 0
...
prompt>
```

Configurando um Sistema de Arquivos

Criando e Montando o FS

A maioria dos FSs fornece uma ferramenta geralmente referida como `mkfs`.

- Recebe como entrada um dispositivo (por exemplo, uma partição de disco `/dev/sda1`) e um tipo de sistema de arquivos (por exemplo, `ext4`) e grava um sistema de arquivos vazio, começando com o diretório raiz nessa partição de disco.
- O comando `mount` (que internamente usa a chamada de sistema `mount()`) recebe um diretório existente como o ponto de montagem de destino (ou seja, o diretório onde o sistema de arquivos é anexado e disponibilizado para o sistema).



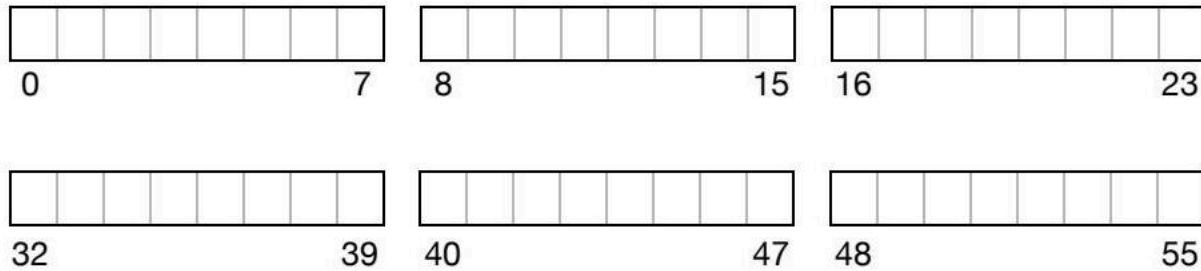
Sistema de Arquivos: Uma Versão Simplificada do Sistema de Arquivos UNIX

Vamos discutir uma implementação simples de FS: o Very Simple File System (vsfs).

- O FS é puro software, ou seja, não requer auxílio do hardware.
 - Mas deve conhecer detalhes do hardware para ser eficiente!
- Existem dois aspectos importantes para um sistema de arquivos:
 - **Estruturas de dados**: quais estruturas em disco são usadas pelo FS para organizar seus dados e metadados.
 - **Métodos de acesso**: como chamadas como `open()`, `read()`, `write()`, etc., são mapeadas para essas estruturas?

Organização Geral do Sistema de Arquivos Muito Simples

O disco é organizado como um array de blocos (por exemplo, 4KB).

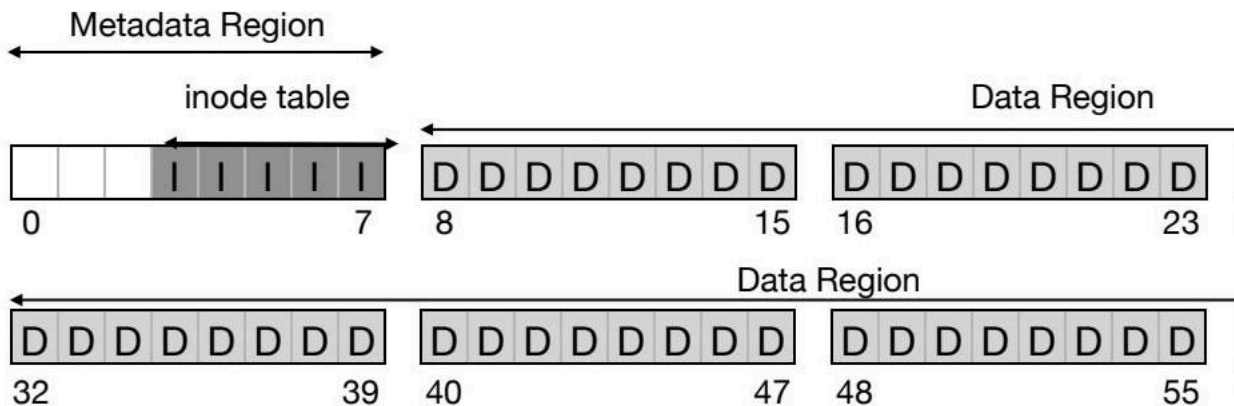


Sistema de Arquivos Muito Simples

Organização Geral

Em um sistema de arquivos muito simples (VSFS), um pequeno disco é organizado como um array de **blocos**, geralmente de 4KB cada. Este disco é dividido em duas regiões principais:

- **Região de Dados:** Armazena o conteúdo dos arquivos.
- **Região de Metadados:** Contém informações sobre os arquivos e o próprio sistema de arquivos.



A imagem ilustra a estrutura detalhada de um sistema de arquivos, compreendendo a Região de Metadados e a Região de Dados. A Região de Metadados contém uma tabela de inodes que abrange os blocos de 0 a 7, enquanto a Região de Dados é dividida em blocos de 8 unidades.

Gerenciamento de Espaço Livre

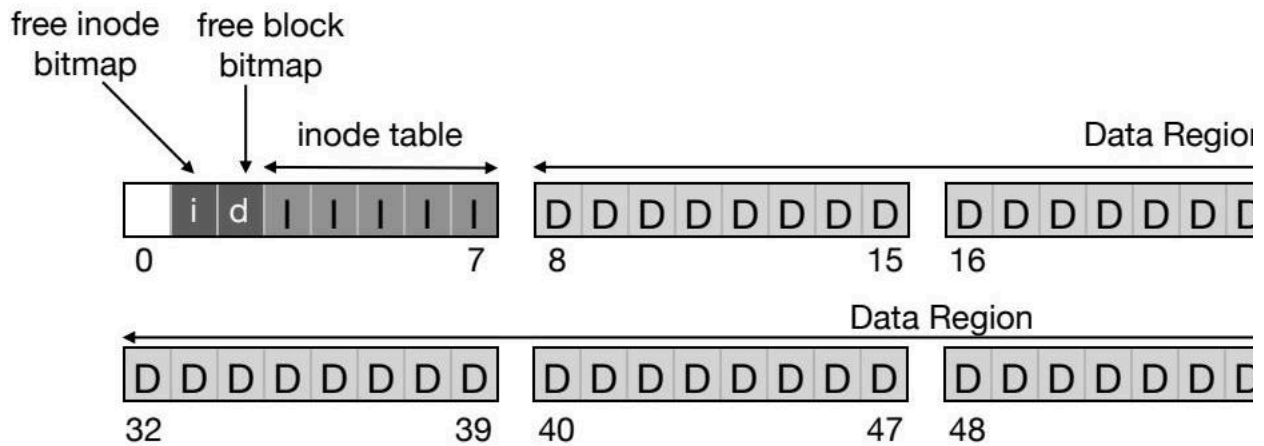
A região de metadados também inclui estruturas para rastrear os **inodes** e **blocos de dados** livres. Um método comum é o uso de **bitmaps**:

- Um **bitmap** para inodes.
- Um **bitmap** para blocos de dados.

Cada bit no bitmap representa se um determinado inode/bloco está em uso (1) ou livre (0). Quando é necessário alocar um novo inode ou bloco de dados, o bitmap é pesquisado para encontrar um espaço não utilizado.

Alguns sistemas de arquivos, como o ext2 e ext3, tentam encontrar uma lista de blocos contíguos para alocar quando um arquivo é criado. Esta estratégia de pré-alocação visa reduzir a fragmentação e melhorar o desempenho.

Outras estruturas são possíveis, como uma lista encadeada onde cada inode/bloco livre aponta para o próximo.



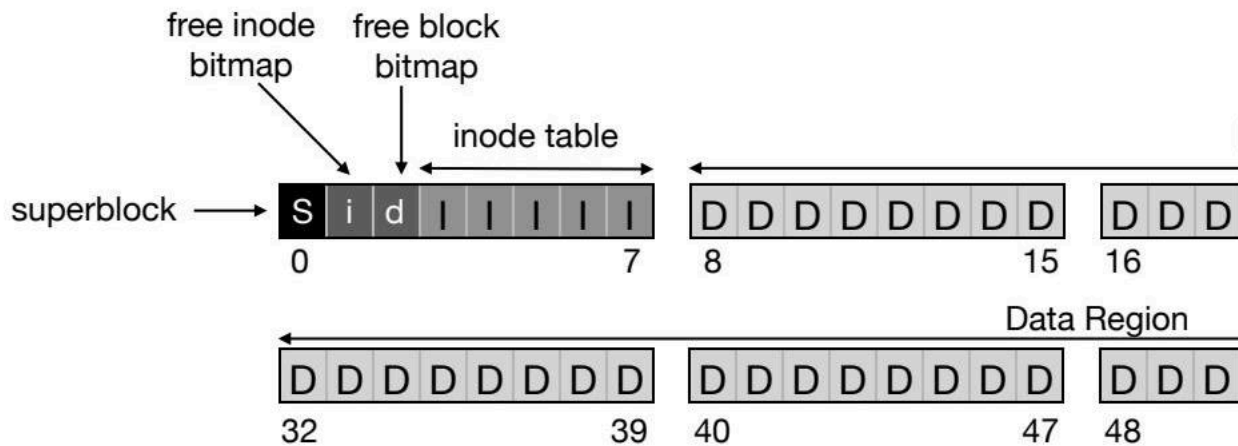
A imagem representa visualmente a organização do armazenamento de um sistema de arquivos, mostrando como os blocos são alocados.

Superbloco

O primeiro bloco na região de metadados contém o **superblock**. O superblock armazena informações essenciais sobre a organização do sistema de arquivos, como:

- Número de inodes e blocos de dados.
- Localização dos bitmaps.
- Localização da tabela de inodes.

Ao montar o sistema de arquivos, o sistema operacional lê o superblock para entender a estrutura do sistema de arquivos.



A imagem fornece uma representação visual de como diferentes elementos trabalham juntos para gerenciar o armazenamento de dados em um sistema de arquivos.

Inode e Tabela de Inodes

Um **inode** (abreviação de index-node) é uma das estruturas mais importantes em um sistema de arquivos. Cada inode é identificado por um número, chamado i-number. A tabela de inodes está localizada em uma região fixa do disco, facilitando a localização rápida das informações para um determinado inode.

Um inode contém metadados sobre um arquivo, como permissões, proprietário, tamanho, timestamps e os ponteiros para os blocos de dados que contêm o conteúdo do arquivo.

Uma visualização da organização geral do disco pode ser descrita da seguinte forma:

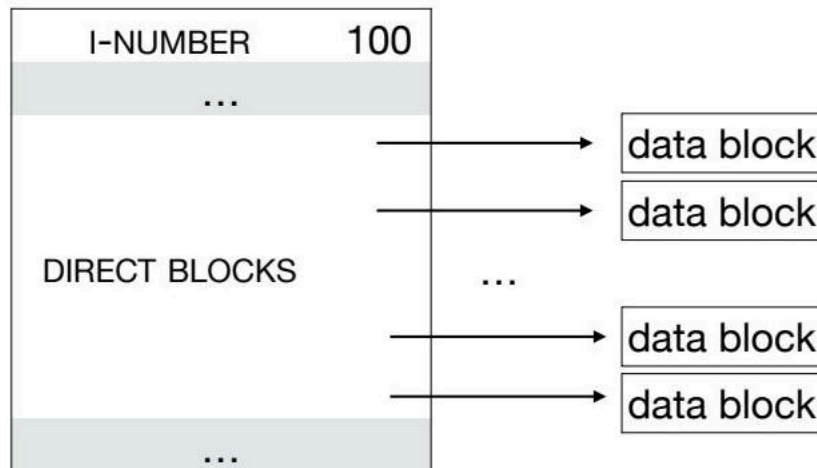
```
| Super | i-bmap | d-bmap | iblock 0 | iblock 1 | iblock 2 | iblock 3 | iblock 4 | ... |
0KB    4KB    8KB    12KB    16KB    20KB    24KB    28KB    32KB
```

Campos de um Inode (ext2 Simplificado)

Campo	Tamanho (Bytes)	Descrição
MODE	2	Permissões de leitura, escrita e execução do arquivo
UID	2	ID do proprietário do arquivo
SIZE	4	Tamanho do arquivo em bytes
TIME (vários)	4	Último acesso, criação, modificação e tempo de exclusão
GID	2	Grupo ao qual o arquivo pertence
LINKS_COUNT	2	Número de hard links para o arquivo
BLOCKS	4	Número de blocos alocados para o arquivo
FLAGS	4	Como o ext2 deve usar este inode
BLOCK	60	Ponteiros para blocos de disco (15 no total)
FILE_ACL	4	Modelo de permissões além dos bits de modo
DIR_ACL	4	Listas de controle de acesso

Ponteiros Diretos para Blocos de Dados

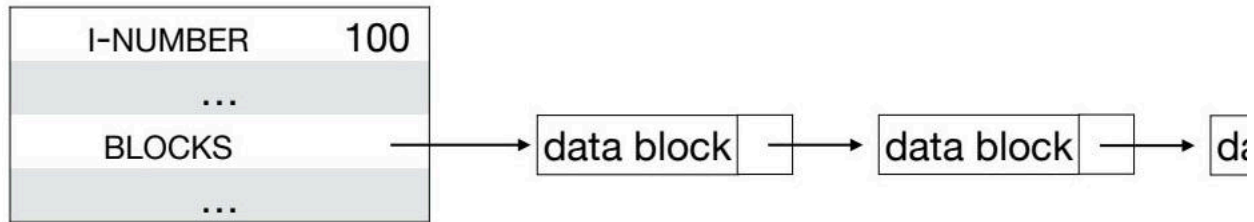
O inode deve apontar para os blocos de dados correspondentes (ou seja, o conteúdo do arquivo). Uma opção é usar **ponteiros diretos**, onde cada ponteiro aponta para o endereço de um bloco de dados. No entanto, para arquivos grandes, o tamanho do inode pode aumentar significativamente, tornando esta abordagem impraticável.



A imagem mostra claramente como os inodes são usados para gerenciar blocos de dados em um sistema de arquivos.

Lista de Blocos

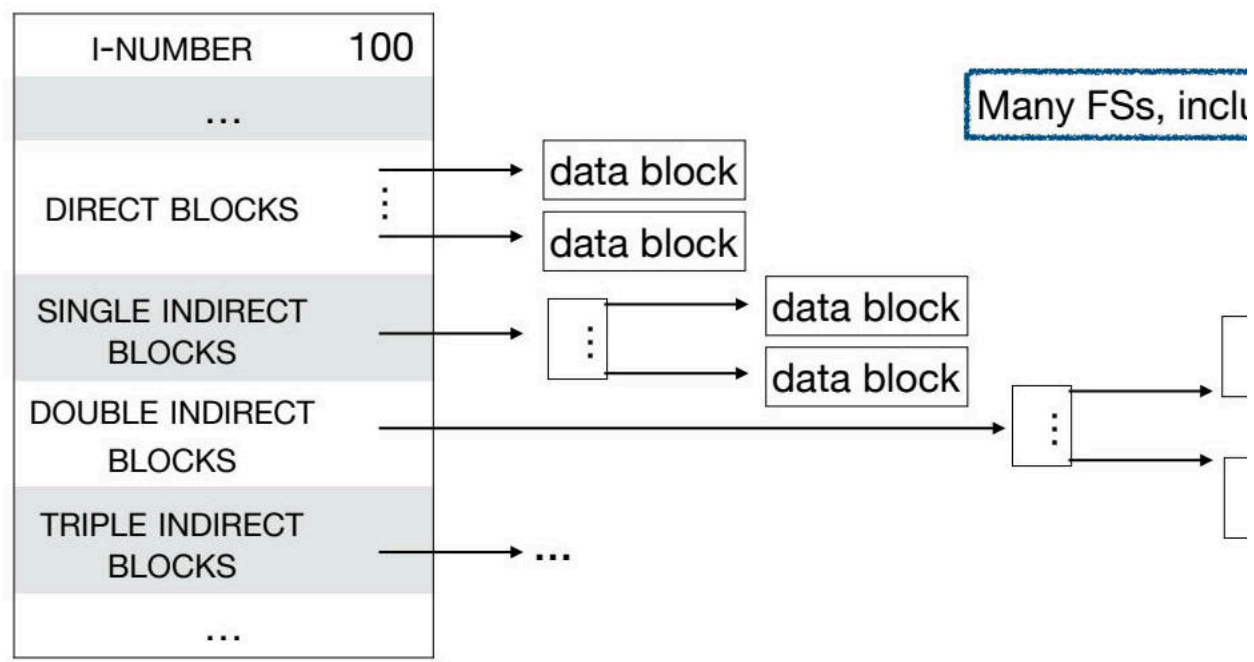
Uma alternativa é o inode conter um único ponteiro para o primeiro bloco do arquivo, e este bloco apontar para o próximo, e assim por diante. Esta abordagem, usada no sistema de arquivos FAT, é ruim para acessos aleatórios, pois para acessar o último bloco, é necessário percorrer toda a lista.



A imagem ilustra a estrutura de lista encadeada, onde um bloco de dados aponta para o próximo.

Ponteiros Diretos e Indiretos Combinados

Outra opção é combinar ponteiros diretos e indiretos. Arquivos pequenos terão ponteiros diretos para todos os seus blocos, enquanto arquivos grandes terão ponteiros indiretos simples, duplos ou triplos para os blocos.

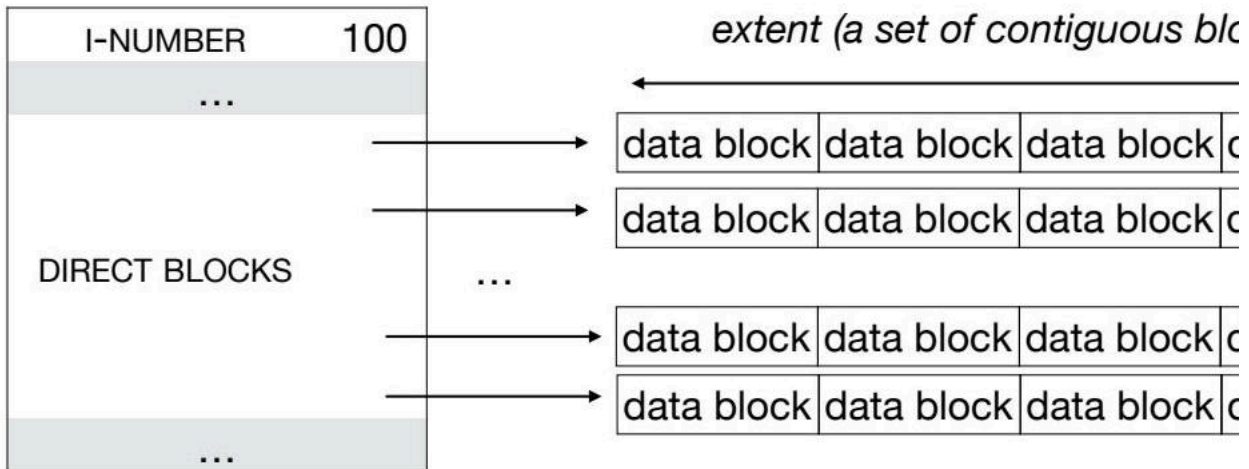


A imagem detalha a estrutura do inode, incluindo blocos diretos, blocos indiretos simples, duplos e triplos.

Utilização de Extensões

Índices multi-nível podem exigir percorrer vários ponteiros para alcançar um bloco, o que penaliza o desempenho. Além disso, alocar blocos para ponteiros indiretos também tem um custo de espaço. Uma alternativa é usar **extents**, que são sequências contíguas de blocos de disco. Isso diminui o número de ponteiros necessários, mas encontrar blocos contíguos nem sempre é fácil.

Extents: Grandes sequências de blocos de disco contíguos.



A imagem destaca o armazenamento eficiente de dados de arquivos agrupando blocos contíguos.

Organização de Diretórios

Os diretórios também têm inodes armazenados na tabela de inodes, marcados como um tipo diferente de inode (diretório em vez de arquivo regular). Os inodes de diretórios também têm ponteiros diretos (ou indiretos) para blocos de dados.

Os blocos de dados de um diretório contêm uma lista de pares (i-number, nome legível pelo usuário).

I-NUMBER	NAME	...	
5	.	...	← the current directory and its contents
2	..	...	← the parent directory and its contents
12	FOO.TXT	...	← i-number of file foo.txt
13	BAR	...	← i-number of directory bar

A imagem mostra a relação entre nomes de arquivos/diretórios e seus i-numbers correspondentes.

Requisição de Abertura de Arquivo

Quando uma requisição para abrir o arquivo `/foo/bar.txt` é emitida, o sistema de arquivos precisa encontrar o inode de `bar.txt`. Para isso, ele deve percorrer todo o caminho:

1. A localização do inode do diretório raiz (`/`) é conhecida. Ao lê-lo, obtém-se o ponteiro para os blocos de dados correspondentes.
2. Ao ler os blocos de dados do diretório raiz, obtém-se o i-number do diretório `foo`.
3. Repete-se o processo para o diretório `foo` para obter o i-number de `bar.txt`.
4. O inode de `bar.txt` é lido na memória, as permissões são verificadas e uma entrada na tabela de arquivos abertos é criada.

Se o arquivo estiver sendo criado:

1. Realizam-se os passos 1, 2 e 3 acima.
2. Lê-se (encontra-se um inode livre) e atualiza-se o bitmap de inodes livres.
3. Escreve-se o novo inode do arquivo no disco.
4. Atualiza-se o bloco de dados do diretório do arquivo.
5. Lê-se e escreve-se o inode do diretório do arquivo.

Requisições de Leitura e Escrita

Uma vez aberto, ler do arquivo `bar.txt` requer:

1. Ler novamente o inode do arquivo (se foi removido da memória).
2. Ler os blocos de dados.
3. Atualizar o inode do arquivo e a entrada na tabela de arquivos abertos.

Escrever em um arquivo pode requerer:

1. Ler novamente o inode do arquivo (se foi removido da memória).
2. Alocar blocos de dados.
3. Escrever os blocos de dados.
4. Atualizar o inode do arquivo.

Otimização de Desempenho de E/S

Caching e Buffering

Abrir, ler e escrever em um arquivo requer várias leituras e escritas no disco. Devido à grande quantidade de E/S no disco, a maioria dos sistemas de arquivos usa intensivamente a RAM. Sistemas de arquivos antigos introduziram um cache de tamanho fixo para armazenar inodes e blocos de dados populares.

Unified Page Cache

Sistemas de arquivos modernos empregam uma abordagem de particionamento dinâmico que integra páginas de memória virtual e páginas do sistema de arquivos em um cache de páginas unificado. As atualizações (escritas) em inodes e blocos de dados também podem ser armazenadas em buffer.

Disk-awareness e Fragmentação

Sistemas de arquivos modernos são disk-aware e colocam inodes e blocos de dados de um arquivo em um grupo de blocos que leva em consideração o layout do disco. O gerenciamento de espaço livre deve ser feito cuidadosamente para evitar a fragmentação.

Desafios de Consistência em Caso de Falhas

Falhas no sistema são esperadas. Garantir um estado consistente do sistema de arquivos em caso de falhas é um problema complexo. Existem múltiplos cenários de falha onde dados e metadados podem ser perdidos ou deixados inconsistentes.

File System Checker e Journaling

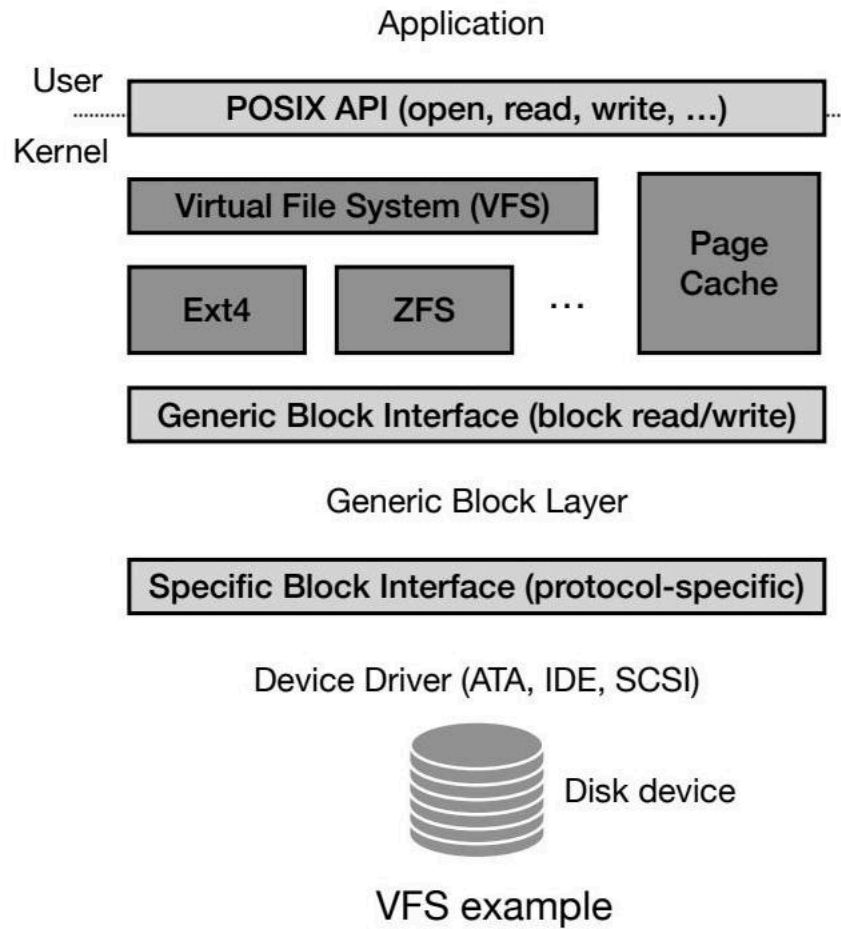
Para lidar com isso, é possível usar um verificador de sistema de arquivos (fsck) ou journaling. O journaling usa um write-ahead log para escrever uma transação antes de atualizar estruturas-chave.

Log-Structured File Systems

Log-structured file systems (LFSs) evitam sobrescrever dados no disco. Eles sempre escrevem em uma porção não utilizada do disco e depois recuperam o espaço antigo através de limpeza.

Virtual File System (VFS)

O Virtual File System (VFS) é uma camada de abstração sobre uma implementação específica do sistema de arquivos. Ele especifica uma interface com operações comuns e facilita a interação com caches do sistema operacional.



O diagrama fornece uma visão geral clara e concisa da arquitetura do VFS e seus vários componentes.